

Due Date – Monday, March 23, 11:59 pm.

Submission Instruction

- (1) Canvas submission will close at 11:59 pm.
- (2) Zip the entire project folder and submit the zipped file to Canvas. The zip file must include the following grading items. **[100 points]**
 - All Java source files in the “src” folder from Project 2, **EXCLUDING** Frontend.java and RunProject2.java.
 - Four JavaFX-related files, which replace the Frontend.java in Project 2
 - **Launcher.java**, containing the **main()** to launch the application.
 - **Main.java**, containing the code for loading the GUI.
 - **Controller.java**, containing the event handlers.
 - **view.fxml**, containing the scene graph (layout of the GUI).
 - Test Specification for functional testing. **[15 points]**
 - Javadoc folder. **[5 points]**
- (3) Do not wait until the last minute to submit the project, as it takes time to upload the zip file. You are responsible for ensuring the project is well-received by the due date and time. **You get 0 points** if you do not have a submission on Canvas. **Projects sent through emails or other methods will not be accepted.**

Project Description

Your team will revamp the interactive user interface in Project 2 and develop a graphical user interface (GUI) for the software system using JavaFX. The GUI shall replace the Frontend class in Project 2. Revamping the user interface does not change the software's functionality. With the new GUI, the software shall behave the same and meet the functional requirements specified in Projects 1 and 2.

Project Requirement

1. Follow the [Coding Standard](#) posted under Modules/Week #1 on Canvas. **You lose points** for violating the rules.
2. You are required to follow the [Academic Integrity Policy](#). See the **Additional Note #14** in the syllabus. If your team uses a repository hosted on a public website, you **MUST set the repository to private**. Setting it to public is considered a violation of the academic integrity policy. The consequences of violation of the Academic Integrity Policy are: **(i) all parties involved receive 0 (zero) on the project, (ii) the violation is reported, and (iii) a record of this violation is in your file.**
3. Each source file (.java file) can only include one public Java class, or you will **lose 2 points**.
4. Include all the Java classes from Project 2 and use them in this project, **EXCEPT** the Frontend class, JUnit test classes, and RunProject2 class. You can reuse the code in the Frontend.java for the Controller.java; however, you cannot directly import the Frontend class and call its methods, or you will lose **10 points**.
5. The TAs will use the same test cases in **Project2TestCases.txt** to grade Project 3. If you lose points in Project 2, you must fix the bugs; otherwise, you will lose points again for the same cause.
6. This project uses the **MVC (Model-View-Controller) design pattern**. You must include **ONLY ONE view.fxml** file for the “View,” **ONE Controller.java** for the “Controller,” and **ALL** the other classes (from Project 2) as the “Model” (the 2 Java packages). In addition, include **Launcher.java** and **Main.java**, which contain the **main()** and **start()** methods for launching and running the GUI application. **You will get 0 points** if you don't follow the MVC design pattern.

7. GUI design.

- If you use only a TextField and a Button to take command lines, **you will get 0 points** for this project.
- The GUI must include TextField, Button, RadioButton, TextArea, TabPane, and GridPane, or **lose 3 points** for each type of GUI object not used.
- You MUST set a “title” for the GUI (the window/stage) or **lose 2 points**.

8. All read (input) and write (output) must be done on the GUI. You CANNOT use **System.out** (write to terminal) or **System.in** (read from terminal) ANYWHERE in any of the classes, **or you will lose 3 points for each violation, with a maximum of losing 10 points**. You should change all the **print()** methods to return strings instead of using System.out to print the data on the terminal.

9. You are required to generate the **Javadoc**. Your Javadoc must include the documentation for the constructors and private and public methods of all Java classes (*.java files). The Javadoc **is worth 5 points**.

- NO need to comment **Main.java** and **Launcher.java**
- MUST comment **Controller.java**.
- DO NOT include the *.fxml files when you generate the Javadoc; **those are NOT Java source files**.
- Generate the Javadoc in a single folder and include it in your project folder for submission.
- You are responsible for double-checking your Javadoc after you generate it. The TAs will navigate the Javadoc with “**index.html**.” You will **lose 5 points** for not including the Javadoc, OR the grader cannot navigate your Javadoc through the “index.html.”

10. Functional Testing.

- Design test cases to test the functionality through the GUI. Test the software as a black box, focusing on the I/O (input/output) behaviors.
- You MUST create a test specification and design at least 15 test cases, in which each test case focuses on a specific objective/functionality. The **Test Specification** is worth **15 points**. You MUST use the table template in the Coding Standard to organize the test cases, or you **will get 0 points** for this part.
- Use the test cases in **Project2TestCases.txt** as a reference to design your test cases and run the tests.
- The GUI shall reject any invalid data and display proper error messages. You will **lose 2 points** for each invalid condition not rejected or each error message not properly displayed.
- You are responsible for thoroughly testing your software beyond the **15 test cases** required. Your software must always run in a sane state and **should not crash in any circumstances**. The TAs will try to produce exceptions while using the GUI. You must handle all exceptions. The software shall continue to run until the TAs stop the execution or close the GUI window. **You will lose 2 points** for each exception not caught.